Ghost API

An AI agent that turns any website into a typed, tested API client

Project specification: explanation, workflow, requirements and build plan

Contents

Part 1: The Simple Explanation
Part 2: Project Overview
Part 3: Users and Use Cases
Part 4: Workflow
Part 5: Requirements (What You Need)
Part 6: Functional Requirements
Part 7: Non-Functional Requirements
Part 8: System Design
Part 9: Output
Part 10: Project Folder Structure
Part 11: Evaluation (Acceptance Criteria)
Part 12: Two-Week Build Plan
Part 13: Risks and Mitigations
Part 14: Why This Project Is Worth Building

Stack: Python, Playwright, free LLM tiers (Gemini, Groq) via LiteLLM, SQLite. **Budget:** $0. **Timeline:** 2-week solo MVP.

Part 1: The Simple Explanation

The restaurant analogy

Picture a restaurant with **no menu**. You can eat there, but only by pointing at other people's plates. Behind the scenes, every time someone orders, the waiter hands the kitchen a neat order slip:

> *"Table 4: 1x Pasta, no cheese, large."*

The kitchen understands those slips perfectly. **You just never see them.**

Ghost API sits in the restaurant, watches every order slip going to the kitchen, and writes the menu for you. After that you can order directly, without pointing at plates.

In real life

- **The restaurant** is any website (a bookstore, a weather site, a job board).
- **Pointing at plates** is how people get data from websites today: copying it by hand, or with fragile "scraper" programs that break whenever the site's design changes.
- **The order slips** are the hidden requests the website makes to its own server. Almost every modern site has them.
- **The menu Ghost API writes** is a small piece of code that lets a programmer get the data directly, cleanly and reliably.

What makes it "AI"

Ghost API doesn't just record. Like a curious person, it:

1. **Explores** the site by clicking, searching and scrolling to see what happens
2. **Notices patterns**, for example "every time I open a book, the site asks for /books/<number>"
3. **Tests its guesses**, for example "if I ask for page 2, do I get different results?"
4. **Fixes its mistakes** when a guess turns out wrong
5. **Remembers** what it learned, so next time is faster

Who would use it

- A **student** who wants data from a site for a project
- A **developer** building an app that needs data from a site with no official API
- A **researcher** collecting public data

What it looks like

The user types one command:

```
$ ghost-api run https://books.example.com --goal "search books and get details"

Exploring site...
   [found] GET  /api/search?q={query}&page={n}
   [found] GET  /api/books/{id}
   [found] GET  /api/books/{id}/reviews
   [test]  Pagination: trying page=2... confirmed
   [test]  Auth: endpoints work without login - OK
Inferring schemas from 14 responses...
Generating Python client...
Running tests... 3/3 passed

Done -> output/books_client.py
```

And gets code they can use straight away:

```
from books_client import BooksClient

client = BooksClient()
results = client.search(query="dune", page=1)   # -> list[BookSummary]
book = client.get_book(results[0].id)           # -> Book (typed!)
print(book.title, book.rating)
```

Part 2: Project Overview

2.1 Problem statement

Websites load their data through hidden internal APIs. Developers who need that data either scrape HTML, which breaks often, or reverse-engineer the API by hand in browser DevTools, which takes hours. No existing tool both **discovers** these APIs and **produces working code** for them.

Approach today	Why it's poor
Scrape the HTML	Breaks whenever the page layout changes, and is slow
Reverse-engineer by hand in Chrome DevTools	Takes hours: clicking around, copying requests, guessing what each field means
Tools like mitmproxy or Postman	Record raw traffic, but don't understand it or write any code

2.2 Solution

An AI agent that drives a real browser, captures network traffic, infers the API's structure, tests its hypotheses by calling the API directly, and generates a **typed, tested client library**.

2.3 Goals

#	Goal
G1	Turn a URL and a plain-English goal into a working, typed client library
G2	Run entirely on free tools and free LLM tiers ($0)
G3	Run on a modest laptop (8 GB RAM, no GPU)
G4	Show transparent reasoning, so the user can see why it did each thing
G5	Improve with memory: re-running on a known site is faster and cheaper

2.4 Non-goals (version 1)

- Bypassing CAPTCHAs, bot protection or paywalls
- GraphQL, WebSocket or gRPC APIs
- Automatic login with stored passwords
- Large-scale data scraping (it generates the client; it doesn't harvest data)
- Mobile apps

Part 3: Users and Use Cases

3.1 User personas

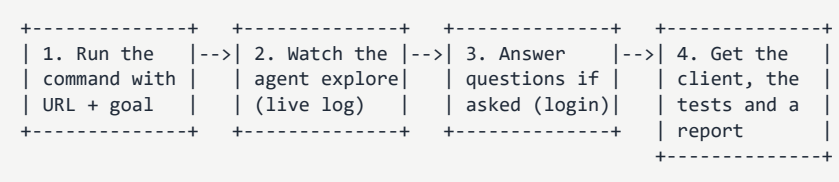
Persona	Need	Example
Student developer	Data for a side project	"I want library book availability in my app"
Indie hacker	Integrate a site with no API	"Pull job listings into my dashboard"
Data analyst	Repeatable data collection	"Track prices on a public catalog weekly"
QA or test engineer	Document an undocumented internal API	"What does our own frontend actually call?"

3.2 Use cases

ID	Use case	Description
UC-1	Generate client	The user gives a URL and a goal, and receives a client library
UC-2	Authenticated site	The user logs in by hand in the opened browser, and the agent continues
UC-3	Refresh client	The site changed; re-run to update the existing client
UC-4	Inspect report	The user reads the discovered endpoints, schemas and the agent's reasoning
UC-5	Export spec	Export an OpenAPI spec for use in Postman, Swagger and similar tools

Part 4: Workflow

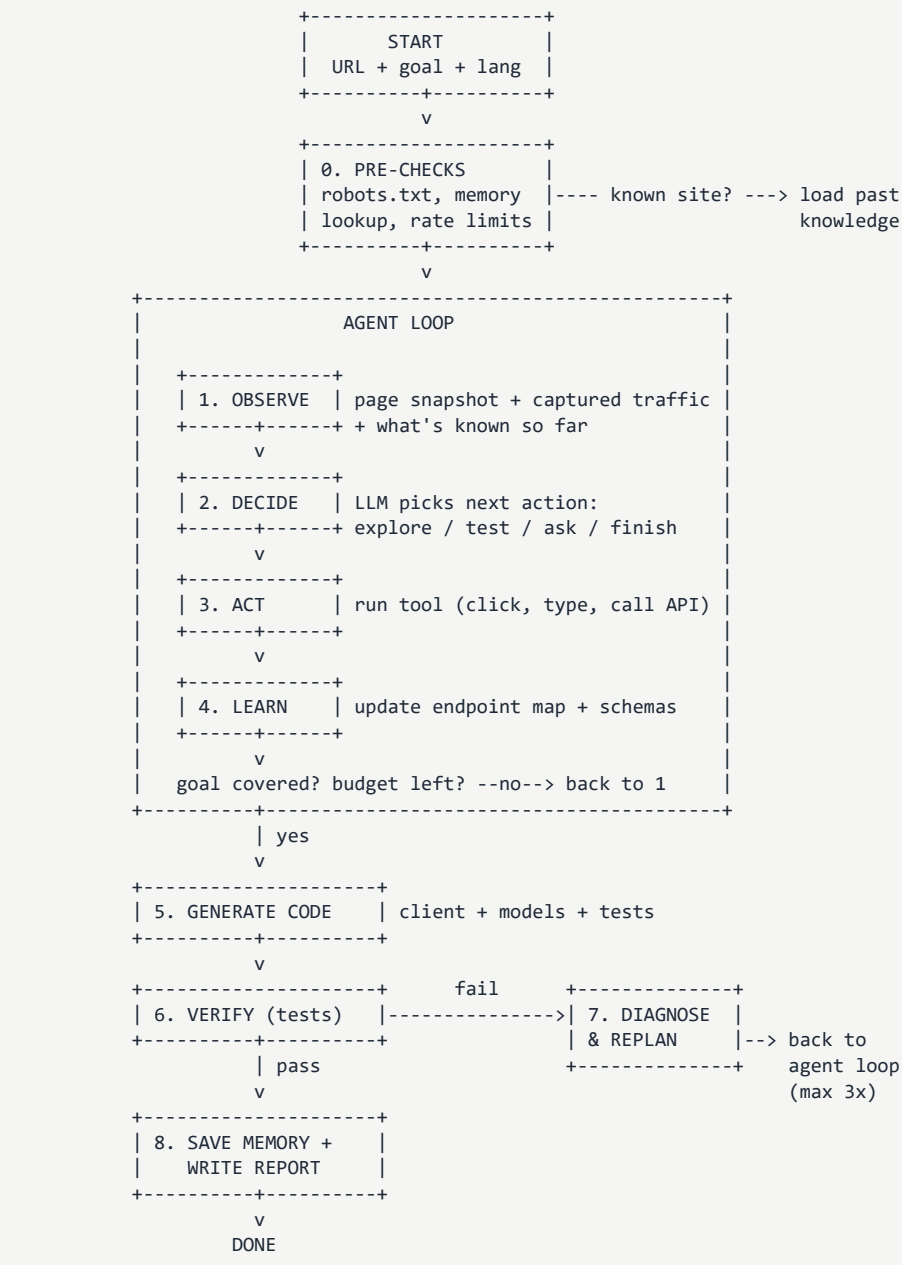
4.1 User workflow (what the user does)



Example command:

```
ghost-api run https://books.toscrape.com --goal "list books and get book details" --lang python
```

4.2 System workflow (what happens inside)



4.3 Each stage in detail

Stage 0: Pre-checks

- Read robots.txt. If the target paths are disallowed, warn and stop unless the user overrides.
- Look the domain up in memory. If it was seen before, load the known endpoints and only re-verify them.
- Initialize the budget: maximum LLM calls, maximum browser actions, maximum time.

Stage 1: Observe

The agent gets a **compact** view so it uses few tokens:

- A simplified page: the list of clickable elements and inputs with short IDs, for example `[12] button "Search"` and `[15] link "Next page"`
- New network requests since the last step (summarized)
- The current endpoint map (what's been found so far)
- Which parts of the goal are still uncovered

Stage 2: Decide

The LLM chooses **one** action from its toolset and explains why:

```
{
  "thought": "I found the book list endpoint but not book details. Clicking a book should trigger it.",
  "action": "click",
  "args": { "element_id": 23 }
}
```

Stage 3: Act

The chosen tool runs (see the toolset in section 8.2).

Stage 4: Learn

This part is **ordinary code, not the LLM**:

- Filters out noise (images, CSS, fonts, analytics, ads)
- Groups URLs into templates: `/api/books/123` and `/api/books/456` become `/api/books/{id}`
- Infers JSON schemas from the responses and merges them across samples
- Marks query parameters as constant or variable

Stage 5: Generate

- The LLM gives endpoints readable names and descriptions
- Code templates produce the client: request functions, pydantic models, an error class and rate limiting
- It also generates a test file and an OpenAPI spec

Stage 6: Verify

- Runs the generated tests against the live site
- Each test calls the endpoint and checks that the response matches the schema

Stage 7: Diagnose and replan

If a test fails, the agent reads the error and decides how to recover:

Failure	Likely cause	Agent's response
401 or 403	Missing cookie, token or header	Retry with the browser's cookies, then find which header is required
Schema mismatch	Too few samples	Explore more to collect samples, then widen the schema
404	Wrong URL template	Re-check how the URL was grouped
429	Rate limited	Back off and slow down
Timeout	Slow endpoint	Retry, then mark as unstable

There are at most 3 replan rounds, then it reports partial success honestly.

Stage 8: Save and report

- Saves endpoints, schemas, auth notes and lessons learned to SQLite memory
- Writes report.md: endpoints found, confidence for each, the agent's reasoning log, and known issues

4.4 Why it's a real agent, not a script

Decision	Example
Where to explore next	"I've found search but not reviews. Click the reviews tab."
When it has seen enough	"Three similar responses. The schema is stable, so stop."
Which guesses to test	"This looks like cursor pagination, not page numbers. Test it."
How to recover from failure	"Direct call returned 403. Maybe a cookie is needed. Retry with the browser's cookies."
When to ask you	"This needs a login. Please log in in the browser window, then press Enter."

Part 5: Requirements (What You Need)

5.1 Hardware

Item	Minimum	Your laptop
RAM	4 GB free	8 GB - OK
CPU	Any modern dual-core	Intel i5-1335U - OK
GPU	Not required	Integrated Intel UHD - OK
Disk	2 GB (browsers plus cache)	OK
Internet	Required	OK

5.2 Software

Software	Version	Purpose
Python	3.11+	Main language
Playwright	Latest	Browser automation
Chromium	Installed via Playwright	Browser
Git	Any	Version control
VS Code	Any	Editor (optional)

5.3 Python libraries

Library	Purpose
`playwright`	Drive the browser and capture traffic
`litellm`	One interface to Gemini, Groq and OpenRouter, with fallback
`httpx`	Direct API calls during testing, and in the generated client
`pydantic`	Data models, both inside the tool and in the generated code
`genson`	Build JSON schemas from sample responses
`jinja2`	Code generation templates
`typer` + `rich`	CLI and nice live terminal output
`pytest`	Run the generated tests
`sqlite3`	Memory and cache (built into Python)
`python-dotenv`	Load API keys from .env

5.4 Accounts (all free)

Account	Why
Google AI Studio	Gemini API key, the main LLM
Groq	Fallback LLM key

Account	Why
OpenRouter	Second fallback (optional)
GitHub	Host the project

Note: free-tier limits change often, so check each provider's pricing page. Free tiers may use your prompts for training, so never send private data.

5.5 Skills you'll use or learn

- Python (intermediate)
- HTTP basics: methods, headers, status codes, JSON
- Browser DevTools Network tab (to check the agent's work)
- Prompting LLMs and structured (JSON) outputs
- Basic agent loop design

Part 6: Functional Requirements

Priority: **M** = must have (MVP), **S** = should have, **C** = could have (later).

6.1 Input and configuration

ID	Requirement	Priority
FR-1	Accept a target URL and a plain-English goal through the CLI	M
FR-2	Accept an output language option (python; typescript later)	M / C
FR-3	Accept budget limits: max LLM calls, max steps, max minutes	M
FR-4	Load API keys from a .env file	M
FR-5	Support headed mode (visible browser) and headless mode	S

6.2 Exploration

ID	Requirement	Priority
FR-6	Open the URL in a real Chromium browser through Playwright	M
FR-7	Extract a compact list of interactive elements (buttons, links, inputs) with IDs	M
FR-8	Let the agent click, type, scroll, go back and navigate	M
FR-9	Stay on the target domain and its API subdomains	M
FR-10	Avoid destructive actions (no "delete", "buy" or "submit payment" buttons)	M

6.3 Traffic capture

ID	Requirement	Priority
FR-11	Record every XHR or fetch request: method, URL, headers, body, status and response	M
FR-12	Filter out non-API traffic (images, CSS, fonts, known analytics domains)	M
FR-13	Store captured traffic in SQLite	M
FR-14	Redact sensitive values (cookies, tokens) in logs and reports	M

6.4 API understanding

ID	Requirement	Priority
FR-15	Group similar URLs into templates with path parameters (/books/{id})	M
FR-16	Identify query parameters and classify them as required, optional or constant	M
FR-17	Infer a JSON schema for each endpoint from multiple samples	M
FR-18	Detect pagination style (page number, offset, cursor)	S
FR-19	Detect authentication style (cookie, bearer token, API key header, none)	S
FR-20	Use the LLM to give endpoints, models and fields readable names	M

6.5 Hypothesis testing

ID	Requirement	Priority
FR-21	Let the agent call endpoints directly (outside the browser) to test guesses	M
FR-22	Test whether headers or cookies are required by removing them one at a time	S
FR-23	Test pagination by requesting the next page and comparing results	S
FR-24	Record each hypothesis and its result (confirmed or rejected)	M

6.6 Code generation

ID	Requirement	Priority
FR-25	Generate a Python client class with one method per endpoint	M
FR-26	Generate pydantic models for request and response data	M
FR-27	Include docstrings with the endpoint description and an example	M
FR-28	Include built-in rate limiting and retry in the generated client	S
FR-29	Generate a pytest test file for each endpoint	M
FR-30	Export an OpenAPI 3 spec	S
FR-31	Generate a TypeScript client	C

6.7 Verification and replanning

ID	Requirement	Priority
FR-32	Run the generated tests automatically after generation	M
FR-33	On failure, classify the error and choose a recovery strategy	M
FR-34	Limit replan rounds (default 3)	M
FR-35	Mark endpoints as verified, unverified or failed in the output	M

6.8 Human in the loop

ID	Requirement	Priority
FR-36	Pause and ask the user to log in by hand when a login wall is detected	S
FR-37	Ask the user when the goal is ambiguous ("I found two search endpoints. Which one?")	S
FR-38	Let the user stop at any time and still get partial results	M

6.9 Memory

ID	Requirement	Priority
FR-39	Save the endpoint map, schemas and auth notes per domain	M
FR-40	On re-run, load memory and only re-verify the known endpoints	S
FR-41	Detect changes (new or removed fields, new endpoints) and update the client	C

ID	Requirement	Priority
FR-42	Cache LLM responses during development to save quota	M

6.10 Reporting

ID	Requirement	Priority
FR-43	Show live progress in the terminal (actions, discoveries, tests)	M
FR-44	Write report.md with endpoints, confidence, reasoning log and issues	M
FR-45	Show usage stats: LLM calls, tokens, time taken	M

Part 7: Non-Functional Requirements

7.1 Cost

ID	Requirement
NFR-1	Total running cost must be **$0** using free LLM tiers
NFR-2	A typical site should need **50 or fewer LLM calls**
NFR-3	Automatically fall back to the next provider when one hits its rate limit

7.2 Performance

ID	Requirement
NFR-4	A typical small site finishes in **10 minutes or less**
NFR-5	Peak memory use **1.5 GB or less** (browser included)
NFR-6	Each LLM prompt **8k tokens or less**, kept small by compressing page and traffic summaries

7.3 Reliability

ID	Requirement
NFR-7	No crash on LLM errors: retry, then fall back, then fail gracefully with partial output
NFR-8	Invalid JSON from the LLM is detected and re-requested (up to 2 times)
NFR-9	A run can be interrupted and resumed from saved state

7.4 Accuracy

ID	Requirement
NFR-10	Endpoint recall **80% or higher** on the test sites
NFR-11	Generated test pass rate **85% or higher**
NFR-12	**Zero invented endpoints**: every endpoint must come from real captured traffic

7.5 Security and privacy

ID	Requirement
NFR-13	Never send cookies, tokens or passwords to the LLM; redact them first
NFR-14	Never store user passwords
NFR-15	API keys only in .env, which is in .gitignore
NFR-16	Generated code must not contain hard-coded secrets

7.6 Ethics and compliance

ID	Requirement
NFR-17	Respect robots.txt by default
NFR-18	Rate-limit its own requests (default: at most 1 request per second)
NFR-19	Never attempt to bypass CAPTCHAs or bot protection; detect them and stop
NFR-20	Send an honest user-agent string identifying the tool
NFR-21	Never click destructive or purchase actions

7.7 Usability

ID	Requirement
NFR-22	One command to run; sensible defaults for everything
NFR-23	Clear, human-readable error messages
NFR-24	Generated code is readable and well named, as if a human wrote it
NFR-25	The README gets a new user running in **5 minutes or less**

7.8 Maintainability

ID	Requirement
NFR-26	Modular design: browser, capture, analysis, agent, generator and memory are separate modules
NFR-27	Adding a new LLM provider means changing only config
NFR-28	Unit tests for the non-LLM logic (URL grouping, schema merging)
NFR-29	Type hints throughout

7.9 Portability

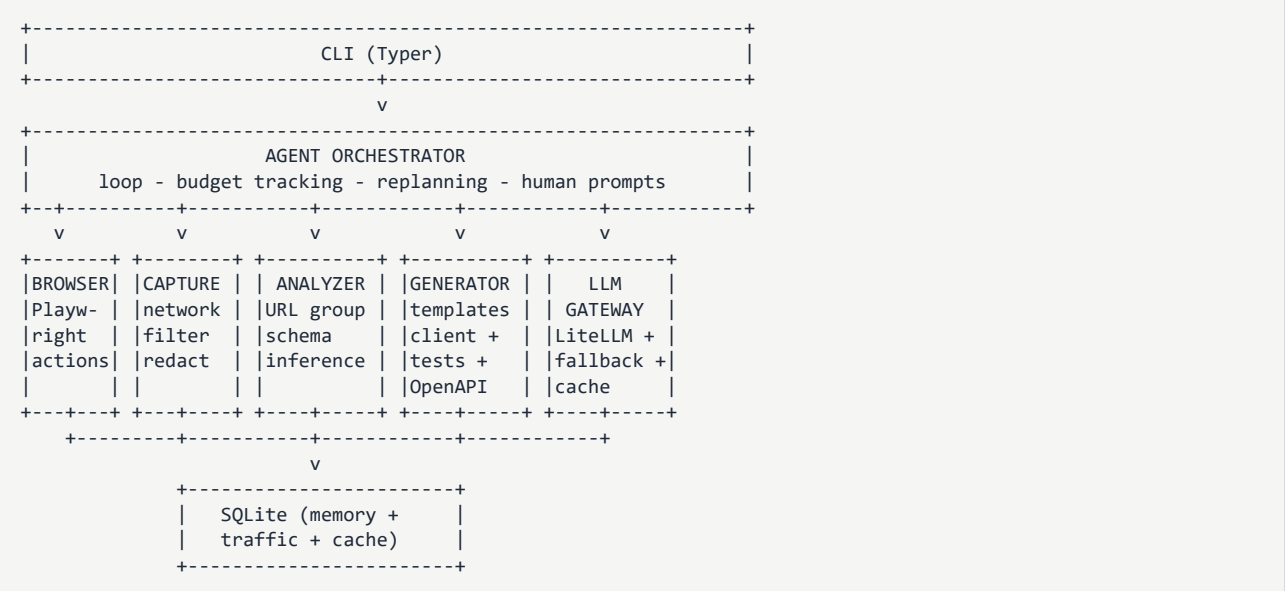
ID	Requirement
NFR-30	Runs on Windows, macOS and Linux
NFR-31	No GPU or paid service required

7.10 Observability

ID	Requirement
NFR-32	Every agent decision is logged as thought, action and result
NFR-33	Runs are saved so they can be replayed for debugging

Part 8: System Design

8.1 Components



Component	Responsibility
CLI	Parses arguments, shows live progress, writes output files
Agent orchestrator	Runs the observe-decide-act-learn loop, tracks budgets, handles replanning and human prompts
Browser	Controls Chromium through Playwright and builds the compact page snapshot
Capture	Records network traffic, filters noise, redacts secrets
Analyzer	Groups URLs into templates, infers schemas, detects pagination and auth (no LLM)
Generator	Turns the endpoint map into client code, tests and an OpenAPI spec using templates
LLM gateway	One interface to all providers, with fallback, retries and a response cache
SQLite store	Holds traffic, endpoints, schemas, memory and the LLM cache

8.2 Agent toolset

Tool	What it does
`click(element_id)`	Click a page element
`type(element_id, text)`	Type into an input
`scroll(direction)`	Scroll to trigger lazy loading
`navigate(url)`	Go to a URL on the same domain
`go_back()`	Browser back
`call_endpoint(method, url, params, headers)`	Call the API directly to test a guess
`inspect_endpoint(endpoint_id)`	View the samples and schema for an endpoint
`record_hypothesis(text, result)`	Log a guess and its outcome

Tool	What it does
`ask_user(question)`	Pause for human input
`finish(reason)`	Stop exploring and move to code generation

8.3 Database tables (SQLite)

Table	Key fields
runs	id, domain, goal, started_at, status, llm_calls, tokens
requests	id, run_id, method, url, status, req_headers, req_body, resp_body, timestamp
endpoints	id, domain, method, url_template, name, description, auth_type, pagination, status
schemas	endpoint_id, kind (request/response), json_schema, sample_count
hypotheses	id, run_id, endpoint_id, text, result, evidence
agent_steps	id, run_id, step_no, thought, action, args, result
llm_cache	prompt_hash, provider, response, created_at

8.4 Memory: how it improves over time

- **Per-site memory:** the endpoint map, schemas, auth style and pagination style for each domain. A second run only re-verifies instead of re-exploring.
- **Lessons learned:** notes like "this site needs the X-CSRF header" or "search only fires after typing 3 characters" are saved and fed back into the prompt on later runs.
- **LLM cache:** identical prompts return cached answers, saving free-tier quota during development.

8.5 Stopping rules

The agent stops when **any** of these is true:

- Every part of the goal maps to at least one verified endpoint
- The step budget is used up (default 40)
- The LLM call budget is used up (default 50)
- No new endpoint has been found in the last 8 steps
- The user stops it

Part 9: Output

```
output/books.toscrape.com/
|-- books_client.py     # The client library
|-- models.py           # Pydantic data models
|-- test_client.py      # Generated tests
|-- openapi.yaml        # OpenAPI spec
+-- report.md           # What was found, confidence, reasoning log
```

Sample report.md excerpt:

```
## Endpoints (3 verified, 1 unverified)
| Endpoint                    | Status     | Confidence | Samples |
|-----------------------------|------------|------------|---------|
| GET /api/books              | verified   | High       | 6       |
| GET /api/books/{id}         | verified   | High       | 4       |
| GET /api/search?q=          | verified   | Medium     | 3       |
| GET /api/books/{id}/related | unverified | Low        | 1       |

## Usage: 31 LLM calls - 42k tokens - 6m 12s - $0.00
```

Part 10: Project Folder Structure

```
ghost-api/
|-- ghost_api/
|   |-- cli.py              # Command-line entry point
|   |-- agent/
|   |   |-- orchestrator.py # Main loop, budget, replanning
|   |   |-- prompts.py      # System and step prompts
|   |   +-- tools.py        # Tool definitions
|   |-- browser/
|   |   |-- driver.py       # Playwright control
|   |   +-- snapshot.py     # Compact page view
|   |-- capture/
|   |   |-- recorder.py     # Network capture
|   |   +-- filters.py      # Noise filtering + redaction
|   |-- analysis/
|   |   |-- url_grouping.py # /books/123 -> /books/{id}
|   |   |-- schema.py       # JSON schema inference/merge
|   |   +-- detectors.py    # Pagination + auth detection
|   |-- generator/
|   |   |-- python_gen.py
|   |   |-- openapi_gen.py
|   |   +-- templates/      # Jinja2 templates
|   |-- llm/
|   |   +-- gateway.py      # LiteLLM + fallback + cache
|   +-- memory/
|       +-- store.py        # SQLite access
|-- tests/                  # Unit tests for non-LLM logic
|-- evals/                  # Test sites + scoring script
|-- .env.example
|-- pyproject.toml
+-- README.md
```

Part 11: Evaluation (Acceptance Criteria)

11.1 Test sites (legal and safe)

- books.toscrape.com and quotes.toscrape.com (built for practice)
- Hacker News (public API to compare against)
- Open Library (public)
- One local demo app you write yourself, so the true answer is known exactly

11.2 Metrics

Metric	How it's measured	Target
Endpoint recall	Found / actual (checked by hand in DevTools)	80% or higher
Endpoint precision	Real / reported	100%
Test pass rate	Passing / generated tests	85% or higher
LLM calls per site	Counted	50 or fewer
Time per site	Measured	10 minutes or less
Cost	Measured	$0

11.3 The MVP is done when

- [ ] One command produces a working client for at least 3 test sites
- [ ] Generated tests pass at or above target
- [ ] The report shows the reasoning log
- [ ] It runs on your laptop with only free API keys
- [ ] The README has a demo GIF

Part 12: Two-Week Build Plan

Day	Task	Deliverable
1	Project setup, Playwright opens a site, captures requests	Traffic printed to the terminal
2	Noise filtering, redaction, save to SQLite	requests table filling up
3	URL grouping and schema inference (no LLM)	Endpoint list from the captured traffic
4	LLM gateway: LiteLLM, fallback chain, cache	llm.ask() works with Gemini and Groq
5	Page snapshot, plus the agent loop with browser tools	Agent clicks around toward the goal
6	call_endpoint and hypothesis testing	Pagination and auth detection
7	Stopping rules, budgets, live terminal UI	Full exploration run
8	Python client and models generator	books_client.py generated
9	Test generator and auto-run	Tests run after generation
10	Diagnose and replan loop	Recovers from 401 and schema errors
11	Memory: re-run uses past knowledge	Second run much faster
12	Report and OpenAPI export	report.md, openapi.yaml
13	Evaluation on test sites, fix the worst failures	Metrics table
14	README, demo GIF, cleanup	Ready to publish on GitHub

Part 13: Risks and Mitigations

Risk	Impact	Mitigation
Free LLM rate limits	Runs stall	Fallback chain, caching, small prompts
Weak free models make bad decisions	Poor exploration	Plain code does the heavy analysis; the LLM only picks actions and names things
Bot protection	Some sites fail	Detect it, stop politely, document as a limitation
LLM invents endpoints	Wrong output	Only endpoints from captured traffic are allowed (NFR-12)
Leaking secrets to the LLM	Privacy issue	Redaction before every prompt (NFR-13)
Legal or terms-of-service issues	Reputation	Practice and public sites only, respect robots.txt, rate limiting
Scope creep	Never finishes	Stick to the M-priority requirements for the MVP

Part 14: Why This Project Is Worth Building

- **It's visibly agentic.** You can show the explore, hypothesize, test and replan loop.
- **Developers would actually use it**, which is good for GitHub stars and your portfolio.
- **It costs $0** and runs on a normal laptop.
- **The demo is clear:** a URL goes in and working typed code comes out.